TRƯỜNG ĐẠI HỌC FPT

EMBEDDED LINUX ASSIGNMENT 02

BÁO CÁO THIẾT KẾ VÀ ĐÁNH GIÁ HỆ THỐNG GIÁM SÁT CẢM BIẾN

Student ID	SE205174
Họ tên	NGUYEN TRAN NAM NGUYEN
Môn học / lớp	LOS201
Giảng viên	VinhPT30
Ngày nộp	26/10/2026

MỤC LỤC & TÓM TẮT

01. Tổng quan hệ thống và môi trường thực hiện

02. Kiến trúc tổng thể; lựa chọn cấu trúc dữ liệu và IPC

03. State machine, luồng dữ liệu và shutdown

04. Thiết kế driver và giao diện người dùng

05. Thread synchronization và phân tích deadlock

06. Race conditions, robustness và limitations

07. Kết quả Task 1 và thu thập dữ liệu

08. Task 2: Valgrind, memory footprint, memory map

09. Task 2: Kiểm tra tích hợp driver

10. Task 3: GDB và strace

11. Task 3: perf và khuyến nghị tối ưu

12. Task 4: SCHED_FIFO, memory lock, absolute sleep

13. Task 4: thống kê jitter và đánh giá ứng dụng

14. Reflection cá nhân – mẫu câu hỏi cần sinh viên hoàn thiện

Executive summary

Dự án triển khai một ứng dụng Embedded Linux gồm bốn thread liên kết với character device /dev/sms_sensor và giao diện thống kê /proc/sms_stats. Trên QEMU ARM, ứng dụng đã đọc được dữ liệu cảm biến và tạo log. Các phép thử độc lập trên Ubuntu host xác minh xử lý SIGINT, bộ nhớ, hành vi hệ thống, profiling và so sánh scheduling jitter. Các kết quả đo phải được diễn giải theo môi trường thực nghiệm tương ứng.

01 | TỔNG QUAN HỆ THỐNG

Mục tiêu

Xây dựng Sensor Monitoring System (SMS) nhằm lấy mẫu nhiệt độ/độ ẩm định kỳ, tính giá trị trung bình trượt, phát hiện cảnh báo, ghi nhật ký và kết thúc an toàn. Hệ thống minh họa tương tác kernel/userspace, POSIX threading, đồng bộ hóa và phương pháp đo hiệu năng.

Môi trường triển khai

Thành phần	Môi trường được sử dụng
Host	Ubuntu Linux x86-64
Target mô phỏng	QEMU vexpress-a9, ARM Cortex-A9
Kernel	Linux 5.15, zImage và vexpress-v2p-ca9.dtb
Root filesystem	BusyBox/initramfs; ứng dụng ARM liên kết tĩnh
Công cụ	arm-linux-gnueabihf-gcc, gcc, Valgrind, GDB, strace, perf

Hai chế độ thực thi

Host (-DHOST_TEST): Sensor Thread sinh mẫu mô phỏng để kiểm thử các thread, memory và profiling bằng công cụ trên Ubuntu. QEMU ARM: ứng dụng mở /dev/sms_sensor, cấu hình bằng ioctl và đọc các bản ghi CSV do kernel driver cung cấp. Không dùng kết quả host để thay thế bằng chứng driver trên QEMU.

02 | KIẾN TRÚC TỔNG THỂ

Các tầng hệ thống

Tầng	Khối xử lý	Kết nối
Kernel	sms_sensor.ko; /dev/sms_sensor; /proc/sms_stats	read(), ioctl(), procfs
Sensor Thread	Đọc/generate CSV; parse mẫu	Circular buffer + mutex/CV
Processor Thread	Trung bình trượt 10 mẫu; phát hiện ngưỡng	POSIX message queues
Alert Thread	Nhận, in và đếm cảnh báo	Alert queue và log queue
Logger Thread	Ghi /tmp/sms.log, rotate theo kích thước	Log queue; tệp tin

Luồng dữ liệu chính

Kernel char driver → Sensor Thread → circular buffer → Processor Thread → (1) alert queue → Alert Thread → stderr và (2) log queue → Logger Thread → /tmp/sms.log.

Các ranh giới đồng bộ

Sensor/Processor phối hợp qua circular buffer có mutex và condition variable. Processor/Alert/Logger trao đổi qua POSIX message queues, tránh để các thread xử lý chính ghi log trực tiếp lên thiết bị lưu trữ.

03 | STATE MACHINE & QUYẾT ĐỊNH THIẾT KẾ

Application state machine

Trạng thái	Hoạt động	Chuyển trạng thái
INIT	Parse CLI; chuẩn bị buffer, queue, driver, threads	Khởi tạo xong → RUNNING; lỗi → cleanup
RUNNING	Đọc mẫu, tính toán, alert, log	Hết duration hoặc SIGINT → SHUTDOWN
SHUTDOWN	Tắt running flag, đánh thức waiters, join, close/unlink	Dọn tài nguyên xong → EXIT
EXIT	In thống kê và trả exit status	Kết thúc

Tại sao chọn circular buffer?

Bộ đệm vòng có bộ nhớ cố định, thao tác push/pop O(1), dễ theo dõi overflow và phù hợp dữ liệu mẫu theo thời gian. Danh sách liên kết cấp phát node liên tục gây biến thiên độ trễ, phân mảnh và tăng rủi ro quản lý bộ nhớ.

Tại sao chọn POSIX message queue cho alarm?

Queue thể hiện thông điệp rời rạc, có cấu trúc và giao diện send/receive rõ ràng, tách thời điểm phát hiện khỏi xử lý alert. Nếu dùng shared memory, cần bổ sung cơ chế đánh thức và đồng bộ riêng để quản lý vòng đời từng cảnh báo.

Tại sao tách Logger Thread?

Ghi file có thể chậm và biến thiên thời gian. Logger độc lập giúp Sensor/Processor không trực tiếp đợi thao tác ghi, đồng thời tập trung việc flush/rotate log ở một nơi.

04 | DRIVER VÀ CLI

Kernel driver

Thiết bị /dev/sms_sensor cung cấp chuỗi mẫu dạng timestamp_ms,temperature,humidity. Giao diện ioctl SMS_SET_INTERVAL thay đổi chu kỳ đọc, mặc định driver 100 ms. /proc/sms_stats cung cấp sampling interval, read_count, mẫu gần nhất và driver uptime.

Luồng tích hợp ứng dụng ARM

Sensor Thread mở thiết bị để gọi ioctl cấu hình, đóng descriptor cấu hình, sau đó theo từng chu kỳ mở/đọc/đóng, parse CSV bằng sscanf rồi đẩy mẫu vào circular buffer. Mã nguồn có nhánh xử lý EINTR, EAGAIN, EOF và đường dọn descriptor sau read; việc phục hồi khi thiết bị mất giữa chừng vẫn có thể cải thiện.

CLI

Tham số	Ý nghĩa
-i	Sampling interval theo millisecond; mặc định 500
-t	Ngưỡng TEMP_HIGH; mặc định 40°C
-h	Ngưỡng HUMID_HIGH; mặc định 80%
-d	Thời gian thực thi, giây; 0 = liên tục
-v	Hiển thị chi tiết từng sample

Logic cảnh báo

TEMP_HIGH nếu nhiệt độ > 40°C; HUMID_HIGH nếu độ ẩm > 80%; TEMP_SPIKE nếu |Δtemperature| > 5°C. Processor Thread tính rolling average của 10 mẫu gần nhất để ghi nhận trạng thái xu hướng.

05 | THREAD SYNCHRONIZATION

Shared data và cơ chế bảo vệ

Dữ liệu	Thread truy cập	Bảo vệ/thiết kế
Circular buffer	Sensor, Processor	Mutex bảo vệ head/tail/count; CV báo có mẫu
Trạng thái running	Main, các worker	Atomic flag để phát tín hiệu shutdown
Sample/rolling average	Processor và luồng log	Truyền bản sao trong message thay vì trỏ chung
Alert counters	Alert Thread	Thread sở hữu bộ đếm; tổng kết cuối phiên
Log file	Logger Thread	Chỉ một thread sở hữu FILE*
Jitter samples	Sensor Thread	Duy nhất một writer; in báo cáo lúc thoát

Lock order & deadlock

Luồng Sensor và Processor chỉ giữ mutex trong vùng thao tác circular buffer, sau đó nhả khóa trước khi gọi IPC/file I/O. Message queues không yêu cầu giữ cùng mutex của buffer; Logger Thread quản lý FILE* độc lập. Thiết kế này hạn chế chu trình giữ-và-chờ giữa nhiều lock. Cần đối chiếu lại mọi nhánh lỗi của phiên bản source cuối cùng nếu muốn chứng minh hình thức tuyệt đối không deadlock.

Resource Allocation Graph (mô tả)

Sensor → [buffer mutex] → release → Processor; Processor → [buffer mutex] → release → [mq send]; Alert → [mq receive] → [mq log]; Logger → [mq receive] → [log file]. Không có cạnh quay về buffer mutex khi đang giữ file lock trong luồng log hiện tại.

06 | RACE CONDITIONS VÀ ROBUSTNESS

Tình huống race condition tiềm ẩn

Khi Main yêu cầu shutdown đúng lúc Processor đang đợi dữ liệu trên buffer, thread đợi có thể không thức dậy nếu chỉ đổi running flag mà không đánh thức condition variable. Cách xử lý đã được kiểm tra qua unit test: đặt trạng thái shutdown dưới mutex và đánh thức thread đang chờ; pop trả về lỗi/dấu hiệu dừng khi queue hết dữ liệu.

Kết quả kiểm thử buffer

Unit test đã ghi nhận produced=4, consumed=1, current_count=3, overflow_count=1; thử push sau shutdown bị từ chối, và blocked waiting thread trả về -1 sau khi đánh thức. Đây là bằng chứng hành vi shutdown của cấu trúc dữ liệu, không thay thế được kiểm thử stress/race toàn diện.

Giới hạn cần ghi nhận

Các nhánh lỗi open/read của Sensor Thread có thể tiếp tục retry nếu device không còn sẵn sàng. Việc mở/đóng mỗi sample tăng syscall overhead, nhưng đã hoạt động trong bài kiểm tra QEMU. Các phép thử chưa chứng minh hoàn toàn không tồn tại race under adversarial scheduling; khi phát triển sản phẩm nên bổ sung ThreadSanitizer hoặc stress tests phù hợp.

07 | TASK 1: KẾT QUẢ HỆ THỐNG

Kiểm thử trên Ubuntu host

Chỉ tiêu	Kết quả 60 giây
Chu kỳ lấy mẫu	500 ms
Số mẫu jitter được báo cáo	121
TEMP_HIGH	20
HUMID_HIGH	20
TEMP_SPIKE	4
Missed deadlines	0
Graceful shutdown	Hoạt động; POSIX queues được dọn

Kiểm thử trên QEMU ARM

Chạy /bin/sms_app -i 500 -d 10 -v thành công với QEMU_APP_EXIT=0. /proc/sms_stats ghi sampling interval 500 ms và read_count=21 sau phép thử ngắn. File /tmp/sms.log có dòng [SAMPLE] và [ALERT].
Đây là kết quả dùng driver thực, khác với HOST_TEST.

Logger

Logger ghi log qua POSIX queue, flush định kỳ khoảng 5 giây, và rotate khi log vượt ngưỡng 1 MiB sang file .1. File host thực tế đã có bản rotate khoảng 1,1 MB; kích thước lớn hơn ngưỡng là có thể do kiểm tra sau khi ghi record.

08 | TASK 2: BỘ NHỚ

Valgrind: kiểm thử 35 giây + SIGINT

Thông số	Kết quả
Duration	35 giây
Lệnh dừng	SIGINT gửi đến PID Valgrind
Exit code	0

Heap allocations/frees	8 allocs / 8 frees
Heap còn lại	0 byte, 0 block
Valgrind error summary	0 errors
Message queues sau shutdown	Trống

Memory footprint trong QEMU

Thời điểm	VmSize (KB)	VmRSS (KB)
T = 5 s	34,548	4
T = 30 s	34,548	4
T = 55 s	34,548	4

Ba điểm đo không ghi nhận tăng trưởng VmSize/VmRSS. Chỉ số VmRSS 4 KB rất nhỏ so với ứng dụng đa luồng và được báo cáo như phép đo của kernel môi trường mô phỏng, không khẳng định toàn bộ footprint vật lý của ứng dụng chỉ 4 KB. Dữ liệu này không thể thay thế memory profiling sâu hơn.

09 | TASK 2: MEMORY MAP & DRIVER

Các vùng nhớ trong /proc/PID/maps

Region	Địa chỉ ví dụ	Vai trò
Text	00010000–0007c000	Mã thực thi
Read-only	0008d000–00090000	Dữ liệu chỉ đọc
Data	00090000–00092000	Biến toàn cục/dữ liệu có thể ghi
Heap	00092000–000b6000	Cấp phát động
Thread stacks	Các vùng rw-p ẩn danh lớn	Stack cho pthread
Main stack	7ec75000–7ec96000	Stack thread chính
vDSO / vvar	7ee6d000+	Hỗ trợ kernel/userspace

ARM binary được static link nên không có mapping riêng cho libc.so và các thư viện động khác. Các vùng guard page được biểu diễn với quyền ---p.

Driver integration 60 giây

Before	After
interval = 100 ms; read_count = 0	interval = 500 ms; read_count = 119

Với 60,000 ms / 500 ms = 120 chu kỳ lý thuyết, số đọc thực tế 119 nằm trong phạm vi 120 ± 10 mà đề yêu cầu. Chênh lệch 1 lần đọc ≈0,83%.

10 | TASK 3: GDB & STRACE

GDB — kịch bản B: TEMP_HIGH 45°C

Breakpoint 1 được đặt tại app/processor_thread.c:106, ngay điều kiện so sánh temperature và threshold. Breakpoint 2 tại app/alert_thread.c:83. Khi theo dõi message: type=TEMP_HIGH, value=45, threshold=40; counter chuyển từ 0 lên 1. GDB báo Inferior exited normally. Log được tạo trên bản host debug, trước một số chỉnh sửa source mới nhất, nên số dòng code có thể thay đổi.

strace -f -c trong 30 giây

System call	Calls	Seconds	% trong syscall
clock_nanosleep	93	0.429487	41.22%
mq_timedreceive	639	0.373272	35.83%
futex	236	0.214281	20.57%
mq_timedsend	105	0.010261	0.98%
lseek	84	0.008053	0.77%
TOTAL	1,283	1.041931	100%

Tổng thời lượng wall-clock là 30 giây; 1.041931 giây là tổng thời gian strace báo cho các syscall, không phải tổng runtime. Có 556 lần syscall báo lỗi, nhiều lỗi có thể là timeout chủ động của mq_timedreceive; muốn khẳng định loại lỗi phải dùng tracing chi tiết.

11 | TASK 3: PERF & TỐI ƯU

CPU sampling

perf đã thu 838 mẫu cpu-clock; Total Lost Samples = 0. Những hotspot nổi bật: _raw_spin_unlock_irqrestore (14,80%), __syscall_cancel_arch_end (11,34%), finish_task_switch (5,85%), pthread_cond_wait (3,22%), và sensor_thread_func (1,55%).

Giải thích call graph

Chuỗi _raw_spin_unlock_irqrestore → try_to_wake_up → wake_up_q → do_mq_timedsend thể hiện một phần chi phí trong đường đánh thức thread và trao đổi message. Cả strace lẫn perf cùng gợi ý synchronization/IPC overhead có ý nghĩa trong workload hiện tại. Không quy 14,80% CPU cho riêng message queue: symbol này cũng có thể xuất hiện trong nhiều đường kernel khác.

Đề xuất tối ưu — chưa benchmark

Đề xuất	Lợi ích kỳ vọng	Rủi ro/điều kiện
Batch log messages	Giảm gửi queue, syscall và wakeups	Tăng độ trễ log tức thời
Giảm verbose output	Ít printf/format float	Giảm khả năng quan sát
Điều chỉnh queue timeout	Giảm polling không cần thiết	Dừng thread có thể trễ
Giảm open/close mỗi sample	Giảm overhead hệ thống	Phải kiểm tra semantics driver

12 | TASK 4: CẤU HÌNH REAL-TIME

SCHED_FIFO priority 50

Ứng dụng yêu cầu sensor thread dùng SCHED_FIFO priority 50. Log xác nhận “Actual policy: SCHED_FIFO” và “Actual priority: 50”. Bài đối chứng được xác minh là SCHED_OTHER priority 0. Việc kiểm tra actual policy tránh gắn nhãn FIFO sai khi quá trình cấp quyền thất bại.

mlockall

Khi chạy FIFO, ứng dụng gọi mlockall(MCL_CURRENT | MCL_FUTURE), và log có thông báo “Memory locked successfully”. Thành công của lời gọi giúp hạn chế page faults do paging; không bảo đảm hard real-time trong kernel thường.

Absolute periodic sleep

Sensor Thread dùng clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, ...) và cập nhật deadline theo bội số chu kỳ. Nếu deadline đã trôi qua, thuật toán ghi nhận missed deadlines rồi nhảy tới deadline tương lai nhằm tránh drift cộng dồn. Mẫu jitter được tính từ actual wakeup trừ planned deadline và giới hạn lưu 1000 quan sát.

13 | TASK 4: PHÂN TÍCH JITTER

Chỉ số	SCHED_OTHER	SCHED_FIFO
Samples	1,000	1,000
Min	62 µs	10 µs
Average	355.32 µs	243.51 µs
Max	593 µs	326 µs
P99	460 µs	312 µs
Missed deadlines	0	0

Histogram

Bin	SCHED_OTHER	SCHED_FIFO
0–100 µs	26	37
100–500 µs	969	963
500–1000 µs	5	0
>1000 µs	0	0

P99 giảm (460−312)/460 × 100% = 32,17%; average giảm 31,47%, max giảm khoảng 45,03%. Kết quả này chỉ phản ánh các lần chạy trên Ubuntu host. So sánh đồng thời thay đổi scheduling policy và memory locking nên không thể quy trọn mức cải thiện cho scheduling.

Ứng dụng công nghiệp

Nếu cần dự báo deadline nghiêm ngặt, phải đánh giá worst-case latency dưới tải có kiểm soát, cân nhắc PREEMPT_RT, CPU affinity/isolation, priority inversion, cgroup scheduling, lock contention và thiết bị thực thay cho kết quả QEMU. Cần đo lặp lại nhiều phiên để đánh giá biến thiên thống kê.

14 | REFLECTION — SINH VIÊN TỰ HOÀN THIỆN

Theo đề, phần Reflection phải thể hiện trải nghiệm học tập thực tế của sinh viên. Các đoạn dưới đây là khung biên tập, không phải lời kể đã được xác minh. Hãy thay nội dung trong ngoặc vuông bằng trải nghiệm của chính bạn trước khi nộp.

Kỹ năng khó nhất

[MÔ TẢ THẬT: kỹ năng bạn thấy khó nhất, ví dụ cross-compilation, driver ABI, synchronization, hoặc GDB đa luồng. Nêu lỗi cụ thể bạn gặp, log/thông báo lỗi, cách tra cứu và thao tác giúp sửa được.]

Nếu thiết kế lại SMS từ đầu

[CHỌN VÀ GIẢI THÍCH: ví dụ giữ một file descriptor lâu dài thay vì open/close mỗi mẫu, định nghĩa trạng thái lỗi driver rõ ràng, bổ sung test cho retry và timeout. Nêu trade-off của quyết định.]

Kỹ năng hữu ích nhất ở vị trí Embedded Linux Engineer

[LIÊN HỆ CÔNG VIỆC: kernel/userspace interfaces, cross-build, gdb/strace/perf, race/deadlock debugging, reliability measurement. Chọn một hoặc hai kỹ năng bạn thực sự thấy có giá trị.]

Một cải tiến kỹ thuật tương lai

[MỘT CẢI TIẾN CỤ THỂ: MQTT publish, dashboard, dữ liệu tồn tại sau reboot, hoặc regression test real-time. Nêu kiến trúc thay đổi, tiêu chí đo thành công và rủi ro.]

Lệnh QEMU

```
qemu-system-arm -M vexpress-a9 -m 256M -kernel output/zImage -dtb output/vexpress-v2p-ca9.dtb -initrd output/initramfs_asgn2.cpio.gz -append "console=ttyAMA0 rdinit=/init" -nographic
```

15 | PHỤ LỤC BẰNG CHỨNG & KẾT LUẬN

Nội dung	Tệp log
Task 1	logs/task1_final_stdout.txt; logs/task1_final_stderr.txt
Valgrind	logs/valgrind_sms_final.txt; logs/valgrind_app_stdout.txt
Memory	logs/memory_footprint.txt; logs/proc_sms_stats_before.txt; logs/proc_sms_stats_after.txt
GDB	logs/gdb_sms.txt
strace	logs/strace_sms_final.txt
perf	logs/perf_sms.txt; logs/perf.data

Jitter	logs/jitter_other_final.txt; logs/jitter_fifo_final.txt

Ảnh thực nghiệm trên máy Ubuntu

Theo kết quả liệt kê thư mục, report/ chứa 01_QEMU_Boot.png, 02_Sensor_Driver.png, 03_Threads_Test.png, 04_Duration_Test.png, 05_SIGINT_Test.png và 06_Logger_Test.png. Những ảnh này hiện chưa được đưa trực tiếp vào bản PDF nháp vì chưa có bản gốc tương ứng được chia sẻ trong phiên tạo PDF. Khi có file, sẽ chèn hình kèm caption và vị trí kiểm thử.

Kết luận

Assignment 02 chứng minh pipeline hoàn chỉnh từ kernel char driver tới ứng dụng đa luồng và nhật ký trên QEMU ARM, đồng thời có bộ kiểm thử độc lập về tài nguyên, syscall và real-time behavior. Số liệu chính: Valgrind 35s/0 errors/0 bytes at exit; driver 119 reads trong khoảng 60s với 500 ms; strace 30s/1,283 syscalls; perf 838 samples; FIFO P99 312 µs so với OTHER 460 µs. Giới hạn chính là kết quả real-time trên môi trường host/QEMU và một số tình huống lỗi chưa được stress-test toàn diện.

LƯU Ý CẦN BỔ SUNG: Trước khi nộp: điền thông tin sinh viên và reflection thật; bổ sung ảnh thực tế; quay video; kiểm tra code headers; xác nhận tên tệp nộp.